



21st International Summer School on Trustworthy Software

June 30 - July 4, 2025, Shanghai, China

John Hughes – Property Based Testing

Exercises

The lectures use PBT tools for Haskell (and occasionally Erlang), programming languages which you probably do not know well. But there are PBT tools for almost every programming language—varying somewhat in quality, but based on the same principles. It will be much more useful to experiment with a PBT tool in your favourite programming language, than to struggle with a language that is unfamiliar.

1. Find and install a PBT tool for your favourite programming language.
<https://en.wikipedia.org/wiki/QuickCheck> lists 58 reimplementations in 40 different programming languages—and it is not complete. Good choices include:
 - a. Haskell: the QuickCheck and quickcheck-dynamic packages on Hackage.
 - b. Python: Hypothesis (<https://github.com/HypothesisWorks/hypothesis>)
 - c. JVM: ScalaCheck (<https://scalacheck.org/>).
 - d. .NET: FsCheck (<https://github.com/fscheck/FsCheck>)
2. Try the tool out testing list reversal.
3. Your favourite language almost certainly provides a library defining a type of *finite maps*, like the BST type in the lectures, providing (at a minimum) an API for inserting keys and values, deleting keys, and looking up keys to get the corresponding value. Install the library (if need be), and if the insertion and deletion operations mutate the data-structure (rather than returning a new version of it) then define an alternative insertion and deletion API that does not. You can achieve this simply by *copying* the data structure first, then inserting into or deleting from the copy. It doesn't matter if the code is not very efficient; we're only going to use it to practice testing a side-effect-free API.
4. Use the PBT tool you installed in part 1 to test a side-effect-free interface to the library you installed in part 3—either its native interface if the data structure is immutable, or the interface you created in part 3 if it is mutable. Use the ideas from lecture 2: define examples of invariants, postconditions, metamorphic properties and model-based properties. Optionally, assuming you find no bugs in the real code, and that you have access to the source code of the library, experiment with *planting* bugs in the library code, and check that your tests can find them.

5. For the next part, you will need a finite map implementation where insertion and deletion *mutate* the map data-structure (this is essentially what the registry is). If your favourite language does not provide such an API, implement one in terms of the side-effect-free interface by defining a global variable to hold the map state, and defining wrapper functions which access, and when necessary update, the global variable.
6. Now you will need to use the state-machine modelling capabilities of your property-based testing tool. Check that it has them: not all tools do, and without them you will not be able to do this part. The interface may be superficially rather different from that discussed in the lecture 4, but the underlying ideas should be the same. Use these capabilities to develop tests in the style of lecture 4 for the side-effecting finite map API you found (or created) in part 5.
7. (Optional) Collect statistics about the tests you are running in part 6, think about whether the distribution is likely to provoke bugs or not, and make improvements to your generators to make bug discovery more likely. Planting a variety of bugs (one at a time) in your library implementation can be an effective way to evaluate how well you have succeeded.